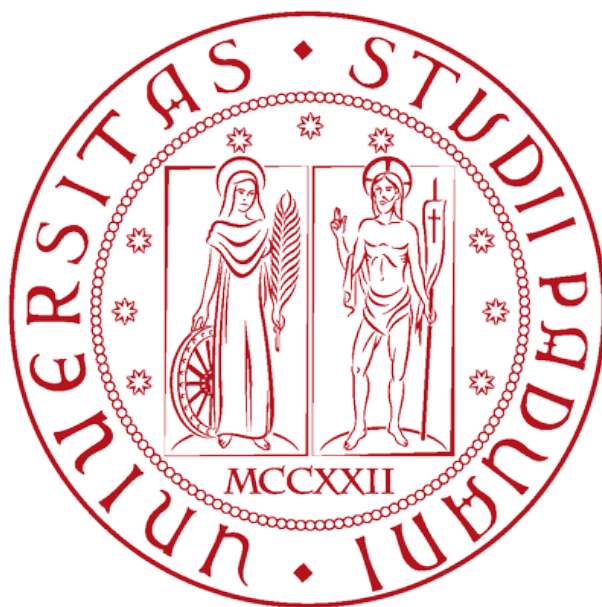


UNIVERSITY OF PADOVA

Embedded Real-Time Control

Laboratory report



Names - (Student ID Numbers)

Academic Year 2021-2022

Contents

1	General notes	2
2	Laboratory 1	4
2.1	SX1509 I/O expander	4
2.2	Line sensor	5
2.3	Keypad	5
2.4	Laboratory work	6
2.5	Conclusion	7
3	Laboratory 2	7
3.1	Timer Configuration	7
3.2	IMU Data Handling	8
3.3	Servo Control	8
3.4	Results	8
4	Laboratory 3	10
4.1	Design process	10
4.2	Theoretical background	10
4.3	Timer Configuration	11
4.4	Implementation	11
4.5	Controller design	13
4.6	Results	13
4.7	Conclusions	14
5	Laboratory 4	14
5.1	Design process	14
5.2	Error definition	14
5.3	Yaw controller Design	15
5.4	Graphs of the Jaw Controller	17
6	Describing the code	19
7	Conclusion	20
8	Used tools	20

1 General notes

This report presents the development, implementation, and evaluation of several embedded control systems for the TurtleBot platform. Over four laboratory sessions, we progressively implemented I2C communication, sensor data handling, servo stabilization, PID motor speed control, and line-following navigation using a custom yaw controller. Each lab builds upon the previous work and introduces increasingly complex real-time control tasks.

This report was created as part of the evaluation for the Embedded Real-Time Control course at UNIPD. All laboratory work and development were performed on a device named TurtleBot (see picture 1). STM32 CUBE IDE was used as the software development tool.

We use a microcontroller from the STM32F7 series, part of STMicroelectronics' STM32 family, based on the ARM Cortex-M7 core, operating at frequencies up to 216 MHz. It is designed for high-performance embedded applications that require substantial processing power combined with real-time capabilities. The STM32F7 features up to 2 MB of dual-bank Flash memory, up to 512 KB of SRAM (with additional TCM memory), and includes an integrated Floating Point Unit (FPU) and Digital Signal Processing (DSP) instructions for efficient signal processing tasks. Key peripherals include high-speed USB OTG (with support for full-speed and high-speed modes), multiple 12-bit ADCs and DACs, timers with encoder and PWM modes, as well as extensive connectivity options such as UART, SPI, I²C, CAN, SDIO, and Ethernet MAC with IEEE 1588 support. The series also includes an L1 cache (I-cache and D-cache), Chrom-ART Accelerator for graphical applications, and flexible memory interfaces for external SDRAM, SRAM, or NOR/NAND Flash. These features make the STM32F7 well-suited for applications involving advanced user interfaces, audio processing, motor control, and industrial automation.



Figure 1: The TurtleBot

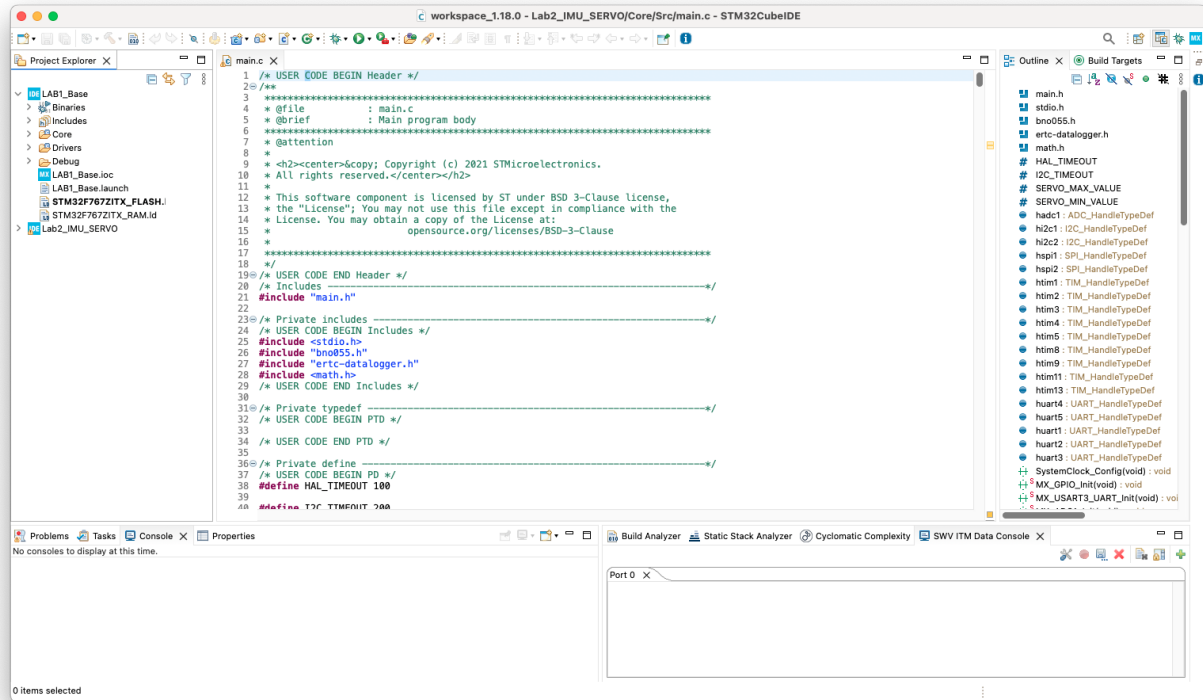


Figure 2: example from STM32 cube IDE

2 Laboratory 1

The goal of the first lab was to establish I2C communication between the STM32 MCU and the SX1509 I/O expander, and to read data from the keypad and the line sensor with the help of this expander. See the configuration in picture 3.

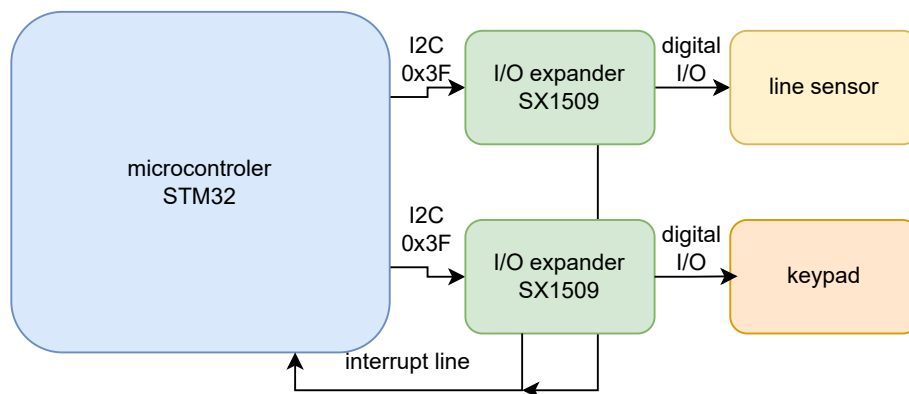
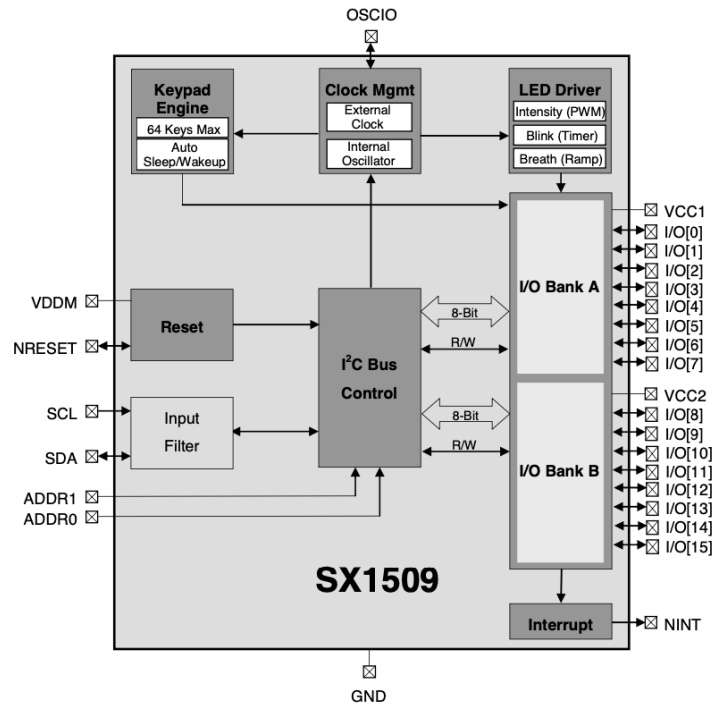


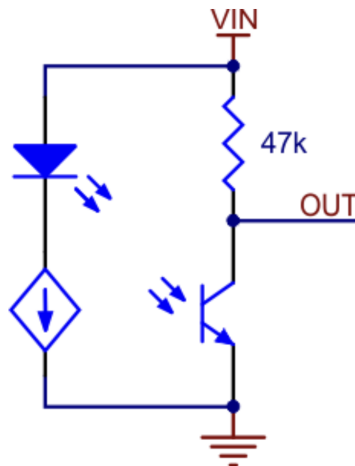
Figure 3: The TurtleBot

2.1 SX1509 I/O expander

We use a 16-channel digital I/O expander SX1509 [1]. The device has two I/O banks, represented by two registers in device memory at addresses 0x27 and 0x28. The I2C address of the device is configurable by two address pins. Up to four devices can be on one line. We use addresses 0x3E and 0x3F since we have two devices. The chip has built-in debouncing, a keypad engine, and an LED driver. See the scheme in picture 4.



2.2 Line sensor



2.3 Keypad

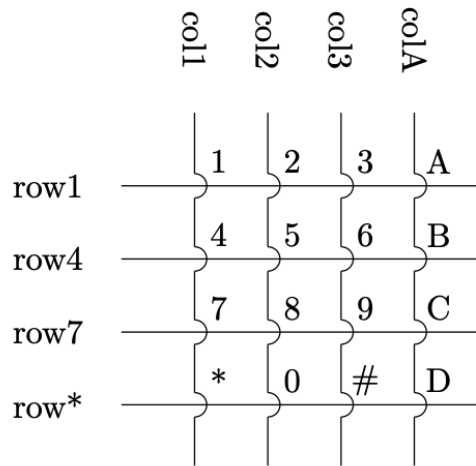


Figure 6: Scheme of keypad connection

2.4 Laboratory work

Exercise 1 & Exercise 2: Write an ISR that recognizes and correctly handles keypad interrupts. You have to properly implement the void HAL_GPIO_EXTI_Callback(uint16_t pin) function. Then print which interrupt has been triggered, i.e., the pin. To be able to receive another interrupt from the keypad, you have to read the registers REG_KEY_DATA_1 and REG_KEY_DATA_2 inside the ISR.

Extend the code of exercise 1 to handle the keypad interrupt. You have to print which keypad button has been pressed.

We have implemented one callback function for reading keypad data only when a button is pressed. We use two global variables: one for the interrupt pin number and one as a flag, so we can execute the appropriate part of code in main. Then we call a function to read data from rows and columns through I2C. See listing 1.

Listing 1: Keypad callback function

```

1  void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2  {
3      IRQ_GPIO_pin = GPIO_Pin;
4      IRQ_GPIO_flag = 1;
5      HAL_I2C_Mem_Read(&hi2c1, I2C_KEYPAD_ADDRESS, KEYPAD_ROWS_ADDRESS,
6      8, key_rows_data, 1, I2C_TIMEOUT);
7      HAL_I2C_Mem_Read(&hi2c1, I2C_KEYPAD_ADDRESS, KEYPAD_COLS_ADDRESS,
8      8, key_cols_data, 1, I2C_TIMEOUT);
9  }
```

To get a character from the keypad, we need to decode the data we received from the expander. For that purpose, we have two functions, as shown in listing 2.

Listing 2: Keypad get cahr functions

```

1  uint8_t GetKeypadPosition(uint8_t data)
2  {
3      if(data == 254)
4          return 0;
5      else if(data == 253)
6          return 1;
7      else if(data == 251)
8          return 2;
9      else
10         return 3;
11 }
12
13
```

```

14     char GetCharFromKeypad( uint8_t const rows, uint8_t const cols )
15     {
16         return keypadLayout[rows][cols];
17     }

```

Exercise 3: Write a routine that reads the status of the line sensor and prints it. The routine must check the status with a polling period of 100ms.

We implemented a function for reading data from the light sensor (see 4), and we call this function by polling in main every 100 ms. We basically just read device registers through I2C.

Listing 3: Keypad get cahr functions

```

1     void GetLightSensorStatus( uint8_t * data )
2     {
3     HAL_I2C_Mem_Read(&hi2c1, I2C_LIGHT_SENSOR_ADDRESS, LINE_SENSOR_ADDRESS,
4     1, data, 1, I2C_TIMEOUT);
5     }

```

Exercise 4: We haven't saved LAB0, since it is not part of the final evaluation, so we cannot extend it. However, our LAB0 was working properly; it was basically just changing the LED delay based on keypad input. That means making a function that returns the delay value based on the input combination.

2.5 Conclusion

In this lab, we established I2C communication with two devices and successfully read data from them. We implemented a keypad decoder, successfully read data from the line sensor, and did not finish exercise 4 because we hadn't saved lab0 on the flash disk and were not able to log in to the school computer for 20 minutes. At least we provided a theoretical description of the problem, since it is quite trivial. Some of the functions are short, so it would make sense to make them inline or not use a function at all. We are not aiming for the most effective code and we prefer readability. Our code was not compiling with inline versions, so we are using regular functions.

3 Laboratory 2

The aim of this laboratory was to implement an open-loop camera stabilization system on the TurtleBot using data from its IMU and controlling two servo motors to adjust the tilt and pan of the camera.

3.1 Timer Configuration

In order to achieve a 100 Hz update frequency, we have the following timer configuration in place:

1. **Prescaler:** The prescaler is used to divide the input clock frequency of 96 MHz by 9600 in order to set the timer's counting frequency to 10 kHz;
2. **Counter Period:** With the counter period of 100, we thus get the desired update frequency of $10kHz/101 \approx 100Hz$.

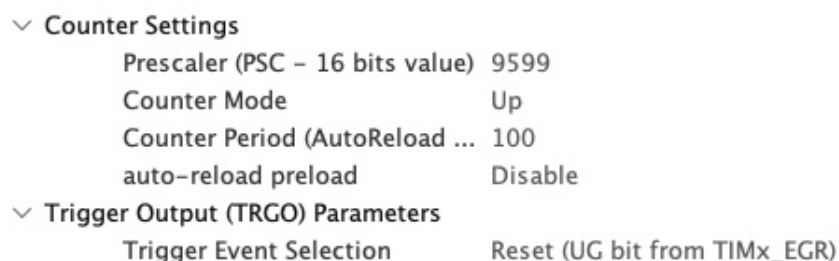


Figure 7: TIM6 Configuration.

3.2 IMU Data Handling

The TurtleBot is equipped with a 6-DOF IMU, consisting of a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures the linear acceleration (m/s^2) along the three axes, while the gyroscope measures angular velocity (rad/s) around these axes. The data from the two sensors is available via

`bno055_convert_double_accel_xyz_msq()` and `bno055_convert_double_gyro_xyz_rps()`, respectively. The data provided by the IMU is crucial when estimating the movement and orientation of the robot, particularly for calculating the tilt as follows:

$$\theta = \sin^{-1}\left(\frac{a_y}{g}\right)$$

where a_y is the acceleration along the y axis.

3.3 Servo Control

The two servo motors controlling the movement are operated via PWM signals generated by TIM6 of the STM32, and the angles are updated with a frequency of 100 Hz using the `__HAL_TIM_PeriodElapsedCallback` function and scaled into a safe duty cycle with the `saturate()` function.

Listing 4: Angle update function

```
1 TIM_HandleTypeDef * timer6_ptr = &htim6;
2 HAL_TIM_PeriodElapsedCallback(timer6_ptr)
3 {
4     theta = asin(d_accel_xyz.y/G);
5     pan = (int8_t)(-(theta/3.14)*180) + PAN_BIAS;
6
7     angle = angle + ((-(d_gyro_xyz.z*0.1)/3.14)*180.)/10.;
8
9     tilt = (int8_t)angle;
10 }
```

As can be seen in the code, there is a strange phenomenon where dividing the angle update formula by 10 ensures a smooth pan without the need for a filter. While this does not make sense time/frequency-wise (after having done the calculations multiple times), it does in the end provide a very smooth movement. The most probable reason is that, by reducing the amplitude of the response, some noise from the gyroscope is actually "filtered".

3.4 Results

Data logging was performed via the UART interface of the STM32, while MATLAB was used to receive and visualize the data in real time with the `serial_data_log()` function. The plots for the yaw compensation and for the tilt compensation can be seen in Figure 8 and Figure 10, respectively.

The data from graphs 8 and 10 was obtained by moving the TurtleBot according to Figure 9 (data for the yaw movement) and Figure 11 (data for the tilt movement), respectively.

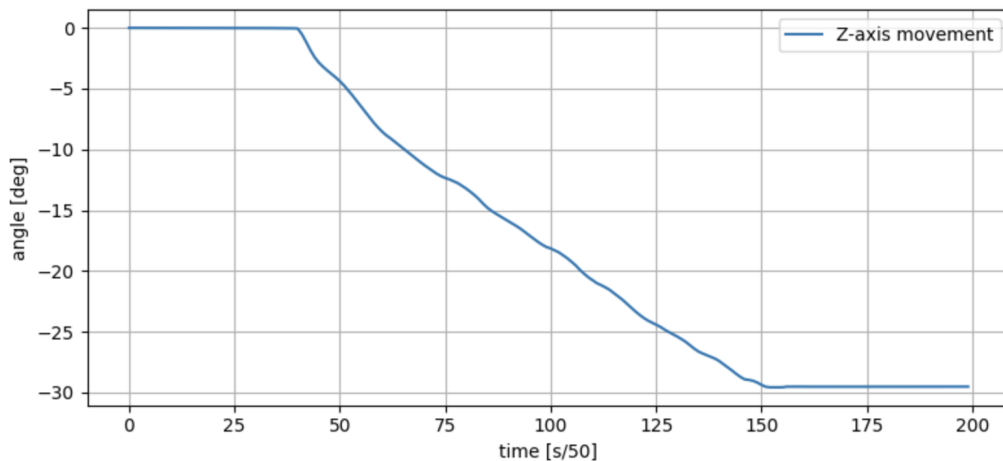


Figure 8: Yaw response behaviour.

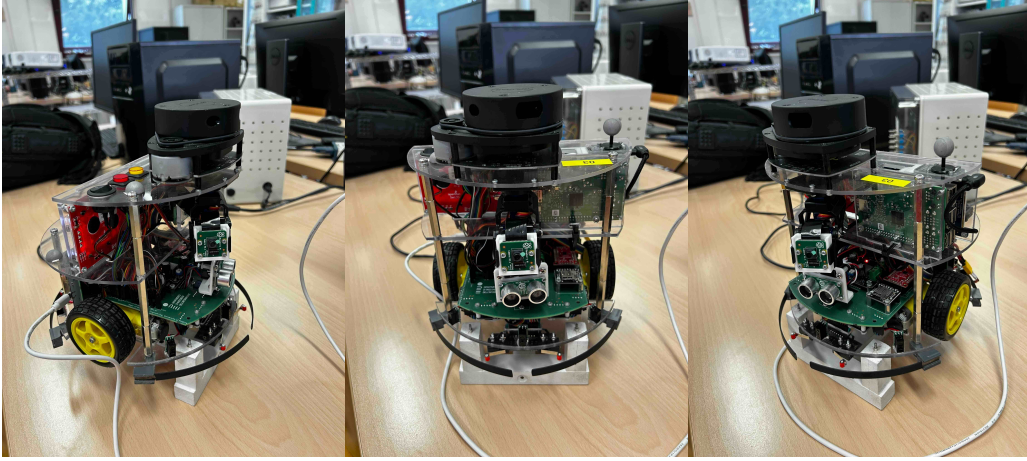


Figure 9: Yaw movement performed during test.

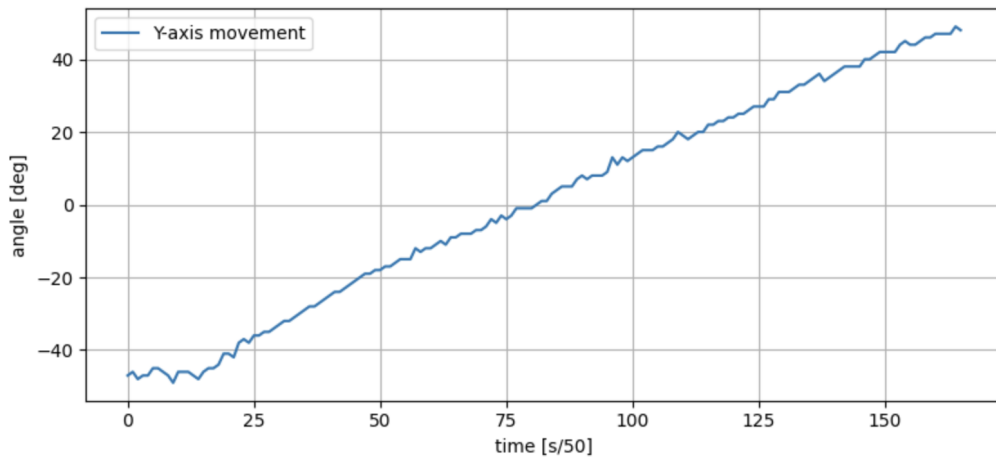


Figure 10: Tilt response behaviour.

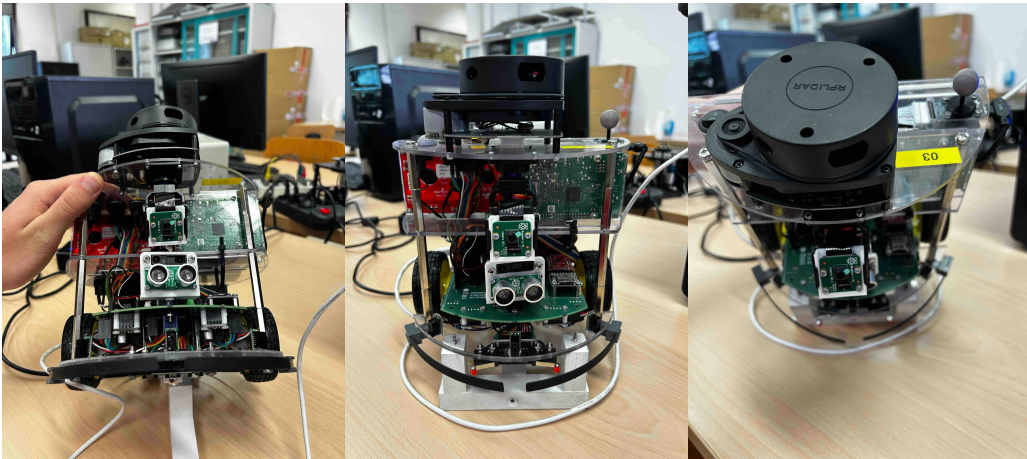


Figure 11: Tilt movement performed during test.

Lab video demonstration::Video Demonstration

The camera stabilization performance was captured and logged while the TurtleBot was tilted manually. As seen in the video, the servo effectively counteracts the tilt to maintain camera alignment, without the need for a complementary filter. In the event of a less optimal camera setup, the complementary filter would have had the following form:

$$\theta_{fused} = \alpha \cdot \theta_{gyro} + (1 - \alpha)\theta_{accel} \quad (1)$$

, where θ_{gyro} and θ_{accel} are two different sensor measurements, from the gyroscope and accelerometer, respectively, thus ensuring a smoother pan.

4 Laboratory 3

In this lab, the task was to implement a PI controller to regulate the speed of the robot's motors. The main objective was to get the wheels to follow a reference value as closely as possible while observing how the controller performed in practice.

4.1 Design process

We implemented the motor controller relatively quickly and tested it. Initially, everything seemed to work fine, so we proceeded with final design adjustments. Unfortunately, we introduced a small change that caused the error to be computed with the wrong sign. This mistake cost us a significant amount of time, as we assumed the implementation was correct—and since it worked intermittently, it was even more misleading. We also implemented the yaw controller but struggled with tuning it properly. Eventually, we discovered the source of the issue. Unfortunately, we ran out of time during the lab session and were not able to tune the PID controller properly.

4.2 Theoretical background

A DC motor changes its speed depending on the voltage we apply and the load it experiences. In order to keep the speed at a set value, we need a feedback controller designed to work properly even when conditions change.

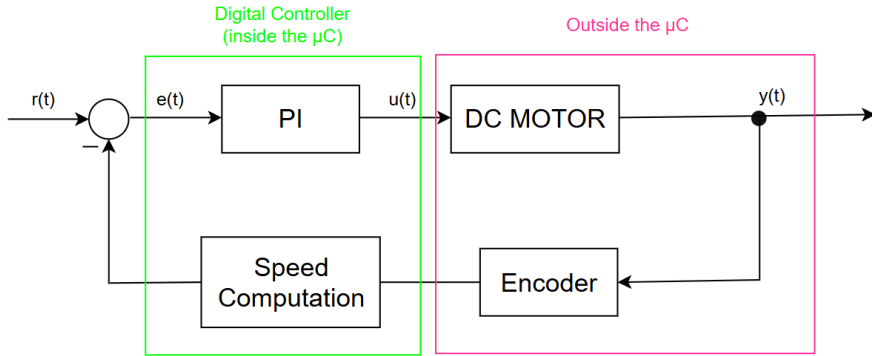


Figure 12: System model

The motor can be easily described by using two equations, one for the mechanical part and another for the electrical side, as follows:

Electrical: From Kirchhoff's Voltage Law (KVL) applied to the armature circuit

$$V_a(t) = R_a i_a(t) + L_a \frac{di_a(t)}{dt} + V_b(t) \quad (2)$$

$$= R_a i_a(t) + L_a \frac{di_a(t)}{dt} + K_b \omega(t), \quad (3)$$

Mechanical: From Newton's second law applied to the rotor

$$J \frac{d\omega(t)}{dt} = T_m(t) - b\omega(t) \quad (4)$$

$$= K_t i_a(t) - b\omega(t), \quad (5)$$

where $V_a(t)$ is the armature voltage (input voltage) [V], $i_a(t)$ is the armature current [A], R_a is the armature resistance [Ω], L_a is the armature inductance [H], $V_b(t)$ is the back EMF (Electromotive Force) [V], K_b is the back EMF constant [V/(rad/s)], $\omega(t)$ is the angular velocity of the rotor [rad/s], $T_m(t)$ is the motor torque developed by the motor [Nm], K_t is the torque constant [Nm/A], J is the moment of inertia of rotor and load [kg·m²], and b is the viscous friction coefficient (damping) [Nm/(rad/s)].

By using the Laplace transform and considering $K = K_t = K_b$, we get the motor's transfer function:

$$G(s) = \frac{\Omega(s)}{V_a(s)} = \frac{K}{L_a J s^2 + (R_a J + L_a b)s + (R_a b + K^2)} \quad (6)$$

Considering the fact that the effects of the inductance are not significant compared to other time constants, the model can be simplified to a first-order approximation.

In our experiment, the transfer function used for tuning the controller had the form:

$$P(s) = \frac{\mu}{1 + Ts} \cdot e^{-\tau s} \quad (7)$$

with $\mu = 3.552$, $T = 0.217$, $\tau = 0.2$. These values represent the system gain, time constant, and a small time delay.

The PWM (Pulse Width Modulation) signal allows us to adjust the motor speed by changing the voltage applied. In this way, the duty cycle varies, the voltage change is not done directly, and speed control is smooth.

4.3 Timer Configuration

In order to achieve a 100 Hz update frequency, we have the following timer configuration in place:

1. **Prescaler:** The prescaler is used to divide the input clock frequency of 96 MHz by 9600 in order to set the timer's counting frequency to 10 kHz;
2. **Counter Period:** With the counter period of 100, we thus get the desired update frequency of $10kHz/101 \approx 100Hz$.

We use oversampling for reading, so we used a 100 Hz reading frequency, then we oversample 10x so the control loop is running at 10 Hz.

▼ Counter Settings	
Prescaler (PSC – 16 bits value)	9599
Counter Mode	Up
Counter Period (AutoReload ...)	100
auto-reload preload	Disable
▼ Trigger Output (TRGO) Parameters	
Trigger Event Selection	Reset (UG bit from TIMx_EGR)

Figure 13: TIM7 Configuration.

4.4 Implementation

From the beginning, we wanted to see and understand how the motor responds in real life and what exactly our controller should do. We used data logging functions in order to receive real values from the system, and once it became clear what we had to control and what exactly our target was in terms of results, we started working on the STM32 board.

Logic:

- implement new **struct** types, such as
 - *motor_attributes_t*
 - *pid_attributes* - stores PID parameters
 - *turtle_bot_attributes_t*
- Encoder reading and speed calculation:
 - *getSpeed()* - reads from the encoder and calculates angular speed
 - *speedRecountToRs()* and *speedRecountToMs()* - convert values from the encoder

- PID controller logic: *pidRecount()* - creates the PID
- PWM duty cycle:
 - *recountDuty()* - converts the controller output into a PWM value
 - *updateMotorSpeedFC()* and *updateMotorSpeedFB()* - apply duty cycle to the motor

The first thing we did was create a structure where we defined the PID controller's parameters. It consists of the proportional, integral, and derivative gain, error terms, and the output. It is important to note that we had to create two instances of this structure, one for the left wheel and one for the right wheel.

Listing 5: PID calculation function

```

1 void pidRecount(pid_attributes_t * pid_attributes, motor_attributes_t *
   motor_attributes)
2 {
3     pid_attributes->error_previous = pid_attributes->error;
4     pid_attributes->error = motor_attributes->wheel_set_speed -
       motor_attributes->wheel_read_speed;
5
6     pid_attributes->derivation = pid_attributes->error - pid_attributes->
       error_previous;
7
8     if(( pid_attributes->integration +100 < 3000) && (pid_attributes->integration
       -100 > -3000))
9         pid_attributes->integration += pid_attributes->error;
10
11     double integral_part = (TS/pid_attributes->Ti)*pid_attributes->integration;
12
13     double derivate_part = (pid_attributes->Td/TS)*pid_attributes->derivation;
14
15     pid_attributes->output = pid_attributes->Kr*(pid_attributes->error +
       derivate_part + integral_part);
16
17 }
```

The function **pidRecount** computes the PID controller, which updates after each timer interrupt. The proportional, integral, and derivative gains for the controller were chosen by experimenting, which means different values were tested on the system until we obtained a satisfactory set of parameters. This set ensures good performance in terms of tracking the reference, as required in the task. Inside *HAL_TIM_PeriodElapsedCallback*, control operations such as PID calculation and PWM update are performed periodically.

The **Forward/Brake** driving method was implemented in order to manage the behavior of the motor. The logic is as follows: the motor rotates forward when the PID output is positive, but if the output is negative, the motor stops.

Listing 6: Fragment from the implementation of Forward/Brake method

```

1 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
2 {
3     // ...
4     if(htim->Instance == TIM6)
5     {
6         data.w1 = motor_attributes_right.wheel_set_speed;
7         data.w2 = motor_attributes_left.wheel_set_speed;
8         data.u1 = line_follower_attributes.distance_from_center;
9         data.u2 = pid_attributes_jaw.output;
10
11         ertc_dlog_send(&logger, &data, sizeof(data));
12         // ...
13     }
14     // ...
```

```

15     updateMotorSpeedFB(&motor_attributes_left, pid_attributes_left.output);
16     updateMotorSpeedFB(&motor_attributes_right, pid_attributes_right.output);
17     // ...
18 }

```

Exercise 2 required us to implement the controller using the Forward/Coast method and make a comparison between its results and those from the Forward/Brake case. Unfortunately, due to time constraints, we were not able to finish this task, and we were not able to properly tune and test the PID controller during the lab. In other words, a direct comparison of the results could not be performed; this remains a task for future work.

4.5 Controller design

Initially, we considered the PI controller approach for this lab, as specified in the laboratory instructions. However, during the implementation, we adopted a PID controller structure to allow for flexible tuning and potentially improved performance.

4.6 Results

This section presents the results of the experiment. To provide a concrete example, we plotted the data collected from the left motor after we implemented the PID on the robot. The recorded values represent the measured angular speed over time and are used to analyze the system's closed-loop response.

We used the following parameters:

- $T_i = 0.1$
- $T_d = 0$
- $K_r = 2$

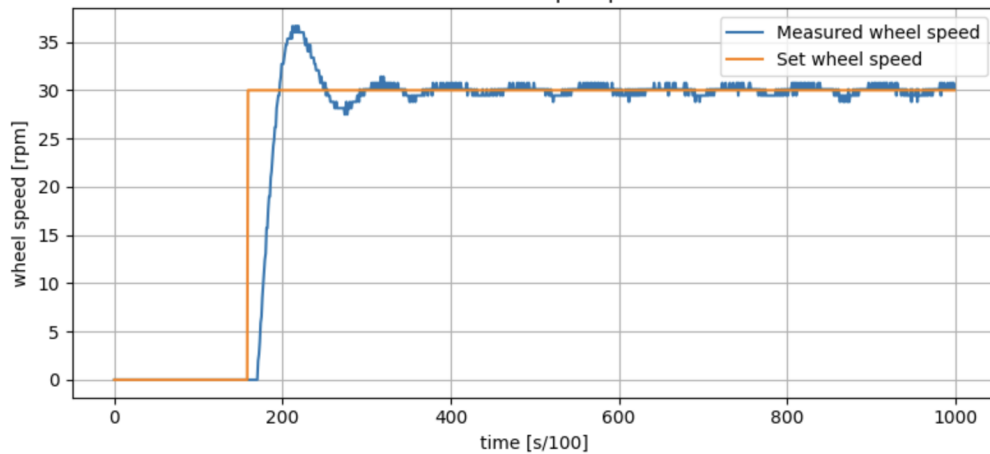


Figure 14: Step response recorded from the left motor

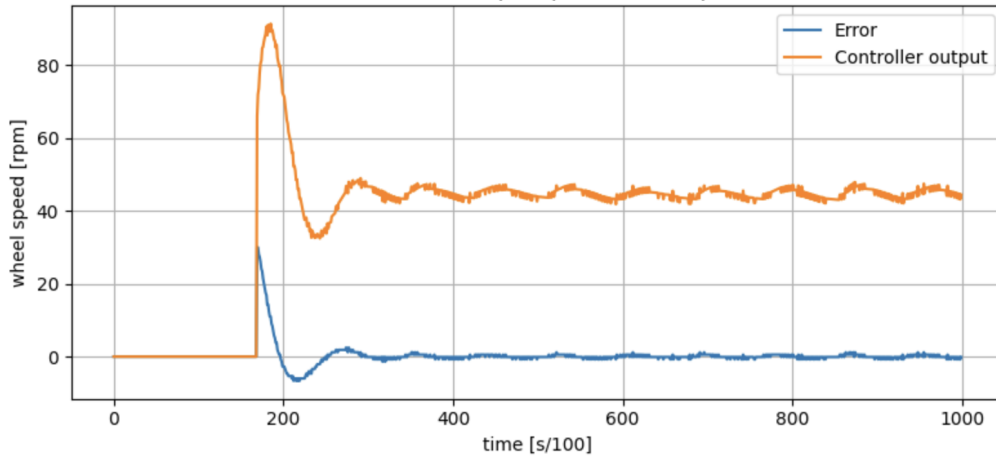


Figure 15: Step response recorded from the left motor PID output

Figure 14 shows the step response of the left DC motor, and 15 shows the output of the PID. The reference value for the angular speed was set to 30 rad/s, and the controller was tasked with tracking this reference as closely as possible. As can be seen from the plot, after the input is applied, the speed increases to an overshoot of 23%. The speed then decreases and oscillates around the reference before settling near the desired value. The collected data allows us to analyze the controller’s performance, but further tuning might be needed to fully eliminate the oscillatory behavior.

4.7 Conclusions

Our implementation was good—the controller was able to track the reference—but there is room for improvement. Different tuning might allow us to get rid of the oscillations, but the main goals were achieved.

To sum up, the implementation managed to get the job done—most of the time, at least. The controller was able to track the speed reference, even though it sometimes took the scenic route before settling down. There’s definitely room for improvement, especially in terms of tuning and getting rid of those small oscillations, but the main goals were achieved. All in all, this lab was a hands-on way to see what actually happens when you go from theory to code, and then to a real motor that doesn’t always behave exactly as expected. Next time, maybe we’ll have more time to fine-tune the controller—and maybe even make that Forward/Coast experiment work as well.

5 Laboratory 4

5.1 Design process

At first, we tried to implement a simple approach, but it did not work as we expected. The idea was to split the left and right sides of the sensor into two separate arrays, giving us two values. Each value represented the occupancy of one half of the sensor. We then subtracted these values to get an error—ideally close to zero.

In hindsight, our implementation may not have been flawed; the real issue was likely the motor control implementation. At the time, however, we assumed the problem was in our logic. We implemented the yaw controller without major issues. It worked as expected.

5.2 Error definition

For starters, let us define our yaw error:

$$\psi_{err} = \psi_{ref} - \psi \quad (8)$$

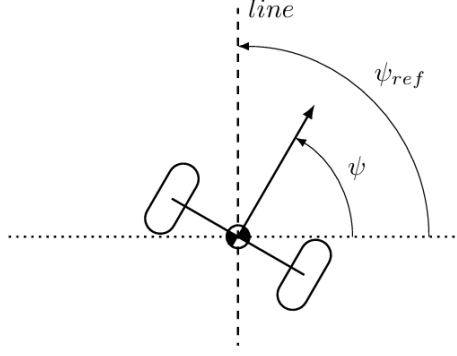


Figure 16: Yaw error definition

As shown in the picture, the error that the yaw controller receives is calculated as the difference between the reference and the current value. We want this error to be minimal. It is also possible to rewrite the equation stated above with respect to the error of following the line, namely e_{esl} .

$$\psi_{err} = \arctan\left(\frac{e_{esl}}{H}\right) \quad (9)$$

where H is the distance between the line sensor and the vehicle's center, equal to 0.085 m, and e_{esl} is defined as:

$$e_{esl} = \begin{cases} 0, & \text{when the line is at the center} \\ > 0, & \text{when the line is on the left side of the sensor} \\ < 0, & \text{when the line is on the right side of the sensor} \end{cases} \quad (10)$$

Therefore, using basic mathematical knowledge, the $\arctan(x)$ is not needed when the value of x is close to 0 (zero). In conclusion, we can define the yaw error as follows:

$$\psi_{err} = \frac{e_{esl}}{H}; \quad (11)$$

We implemented the yaw error as follows:

Listing 7: Yaw error

```

1
2  trurtle_bot->TB_jaw_error = (line_attributes->distance_from_center/LINE_SENSOR_DISTANCE)

```

5.3 Yaw controller Design

For the design of the Yaw controller we implemented the following structure in Matlab Simulink:

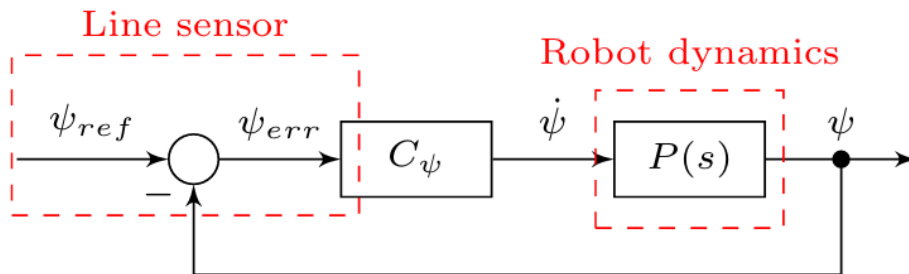


Figure 17: Yaw controller

For the controller, namely C_ψ , it is multiplied by the error that we have defined above. Taking this into consideration, we defined the yaw controller by implementing a proportional (P) control. The value of the yaw controller is defined by the variable K_r .


```

1
2  pid_attributes->output = pid_attributes->Kr*(TB->TB_jaw_error);
    
```

With everything defined, we can introduce the Simulink model we implemented:

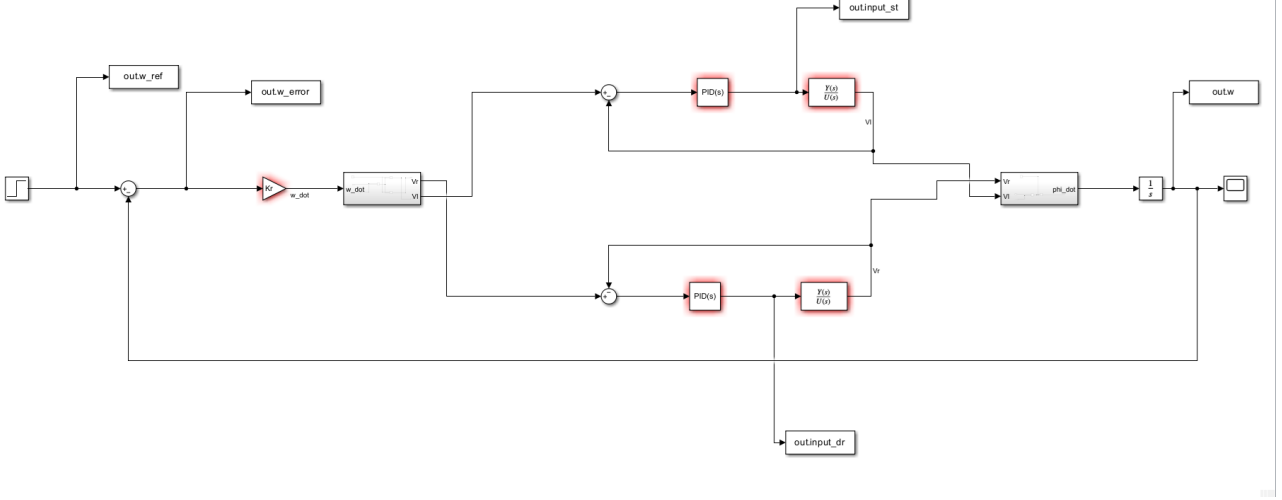


Figure 18: Simulink model

We can clearly see both the PIDs that are intended for controlling the speeds of both wheels. In the effort to find a suitable PID for both wheels such that the overall error is minimal, we implemented a metaheuristic method using the MATLAB function PSO. PSO stands for Particle Swarm Optimization, and it is a method in which we can find the parameters of both the PIDs and the yaw controller gain by running the simulation multiple times and placing particles throughout the search space.

The particles have unique attributes, such as:

1. knowledge of their position in the search space
2. knowledge of the swarm's position in the search space
3. knowledge of their value
4. knowledge of the swarm's values

The goal of this metaheuristic is to find the global optimal value that satisfies the condition given by the user. The condition given is that we want the overall error to be minimal. This can be described as:

$$J = \|\psi_{error}\|; \quad (12)$$

In this way, the goal of the algorithm is to move the particles in the search space at every iteration. Then, by running the simulation with the values obtained in that iteration, the particles obtain an output that is normalized for easier comparison. In the end, after multiple iterations, the particles should gather in a small area, which in the end is the global optimum value, namely the minimal error found in the designated search space.

For the algorithm to work properly, we implemented different intervals that the values should take. Therefore, we can run the same algorithm with multiple choices, building multiple configurations.

The preset configurations are :

1. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; ai, (ai + bi)/2; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
2. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; ai, (ai + bi)/2; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
3. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; (ai + bi)/2, bi; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
4. $param_{ranges} = [ap, (ap + bp)/2; ap, bp; (ai + bi)/2, bi; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
5. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; ai, (ai + bi)/2; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$

6. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; ai, (ai + bi)/2; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$
7. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; (ai + bi)/2, bi; ai, bi; ad, (ad + bd)/2; ad, bd; ar, (ar + br)/2;];$
8. $param_{ranges} = [(ap + bp)/2, bp; ap, bp; (ai + bi)/2, bi; ai, bi; (ad + bd)/2, bd; ad, bd; ar, (ar + br)/2;];$

a_p and b_p are the intervals for the Proportional gain, a_i and b_i are the intervals for the Integral gain, a_d and b_d are the intervals for the Derivative gain and a_r and b_r are the intervals for the Yaw controller gain.

After simulating all the configurations, we have come up with these values:

1. $K_{left}^p = 5,46$
2. $K_{right}^p = 5.193$
3. $K_{left}^i = 51.276$
4. $K_{right}^i = 47.853$
5. $K_{left}^d = 0.505$
6. $K_{right}^d = 0.488$
7. $K_r = 1095$

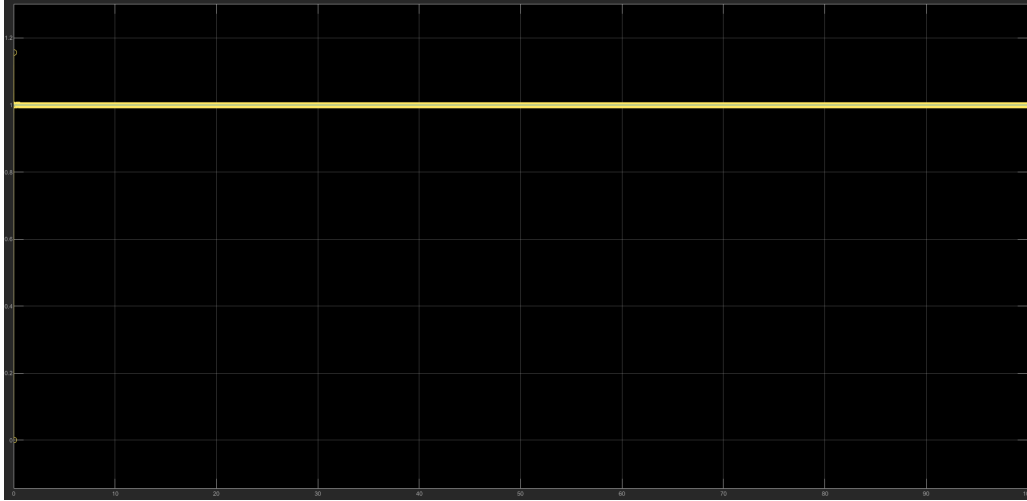


Figure 19: Output with Input

As we can see, the output perfectly tracks the step reference with these values. Taking this into account, we consider it a success in tracking the step reference. However, when testing these values on the bot, we could not get the results we were hoping for. This is likely due to the reference being different or the sensor not reading as the simulation intends.

5.4 Graphs of the Jaw Controller

Going through the simulations, we have gathered data from the rover. We can present the graph for the yaw controller implemented. We have the following graphs:

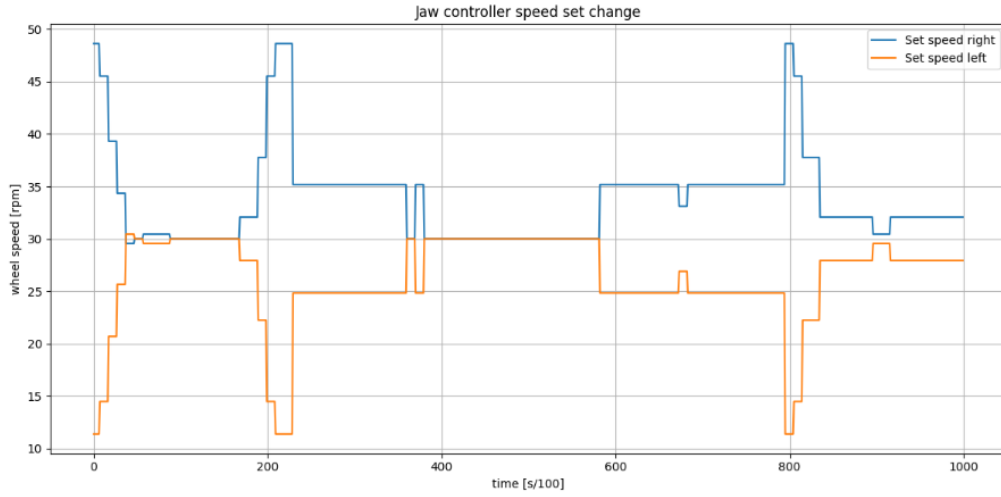


Figure 20: Jaw set speed

The graph illustrates how the yaw controller adjusts the set speed values for the left and right wheels during the line-following task. Periods where both wheel speeds are nearly equal correspond to straight movement along the trajectory. Large differences between the set speeds indicate turning maneuvers or significant corrections, where the robot pivots or curves to realign itself with the path. Sudden jumps and rapid changes in the set speed values correspond to moments of intense correction, possibly due to sharp turns or deviations from the line. Such behavior may also suggest that the controller parameters could be further tuned for smoother motion.

The graph contains the following:

1. X-axis: time, expressed in s/100 (tenths of a second)
2. Y-axis: wheel speed, measured in revolutions per minute (RPM)
3. Set speed right (blue): set speed for the right wheel (RPM)
4. Set speed left (orange): set speed for the left wheel (RPM)

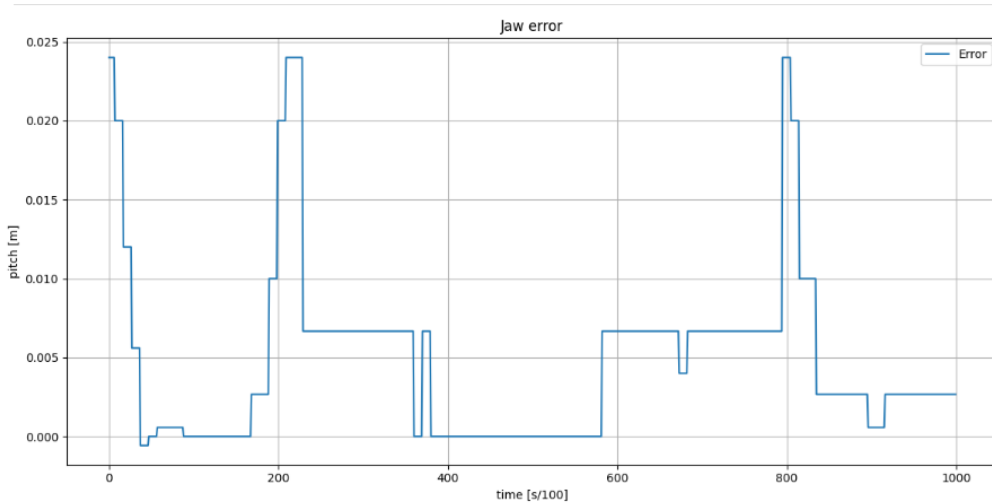


Figure 21: Jaw error

The figure above illustrates the yaw error evolution over time during the robot's line-following task. The graph shows several intervals where the yaw error spikes to approximately 0.025 m (2.5 cm), corresponding to moments when the robot deviates more significantly from the trajectory. Between these events, the error returns to values close to zero, indicating the controller's ability to correct the robot's position. Prolonged intervals of low error indicate good tracking performance, while the spikes suggest moments of trajectory correction or challenging segments in the path.

Overall, the yaw controller is effective at minimizing lateral deviation most of the time, but further tuning may be required to reduce the amplitude or frequency of these error spikes, thus achieving smoother and more robust path following.

The graph contains the following:

1. X-axis: Time, measured in hundredths of a second ($[s/100]$), spanning from 0 to 1000 (0 to 10 seconds).
2. Y-axis: Lateral error (pitch [m]), representing the distance from the center of the line in meters, ranging from 0 to 0.025 m.

6 Describing the code

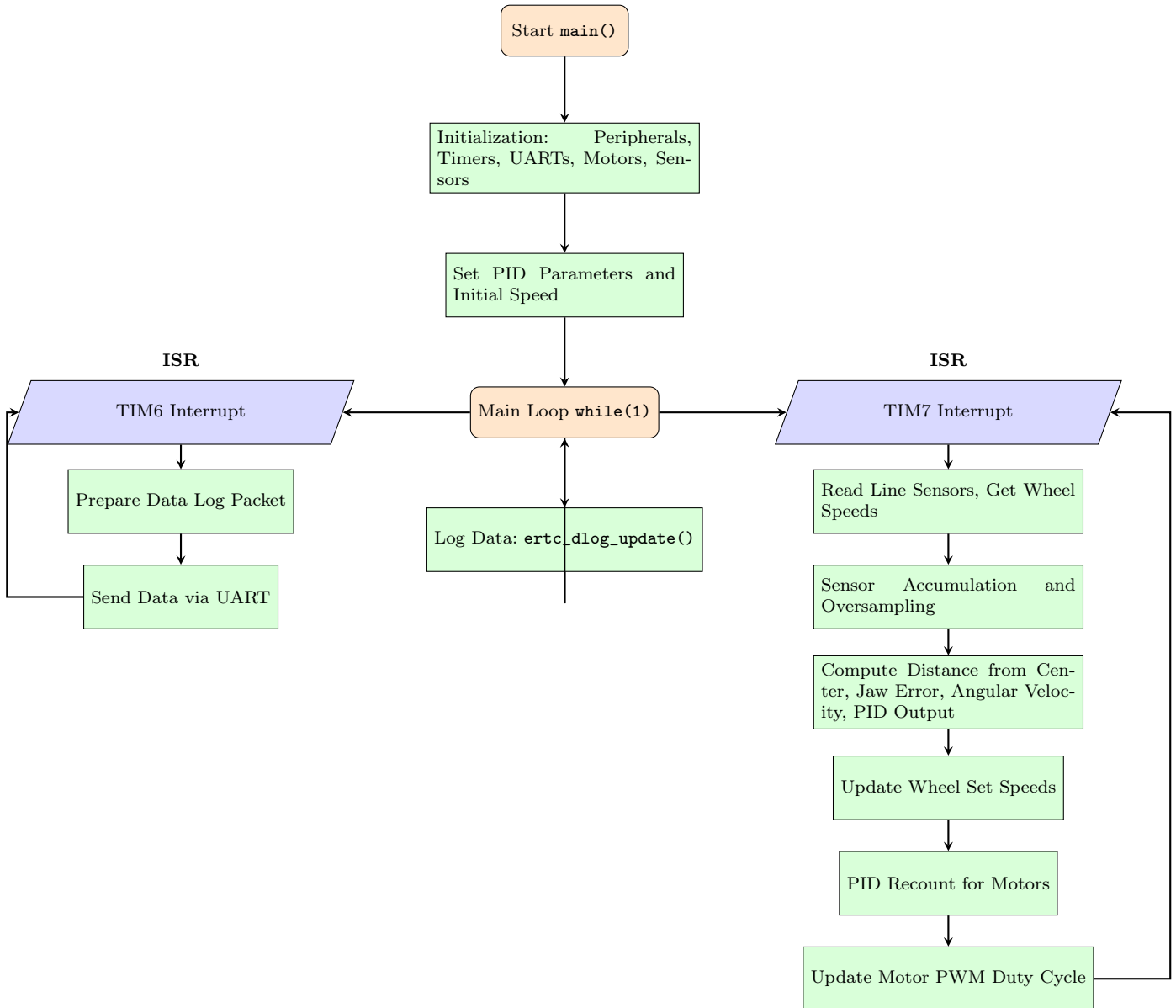


Figure 22: General flow chart for the main structure of the TurtleBot embedded software.

This flowchart represents how the code works. We can further expand this chart by summarizing what each node does in particular. Taking this into consideration, we can define the starting points of the program as follows:

1. **Start main()**: This node represents the program entry point. The function `main()` is called automatically when the controller starts.
2. **Initialization: Peripherals, Timers, UARTs, Motors, Sensors**: Here, all hardware peripherals are initialized.

3. **Set PID parameters and Initial Speed:** The PID controller constants (K_r , T_i , T_d) for both wheels and the yaw controller are set. The initial desired speed of the robot is also set.
4. **Main loop:** The main loop only updates the data logger.
5. **Log Data:** $ertc_{update}^{dlog}()$: Within the main loop, this function is called to update the state of the data logger, handling any data transmission or storage as needed.

After the main loop is triggered, the program enters two different interrupts: TIM6 and TIM7. We can define TIM6 as follows:

1. **TIM6 Interrupt:** This node represents the TIM6 interrupt service routine (ISR), which is triggered periodically based on the timer's configuration. It is used for periodic logging and status signaling.
2. **Prepare Data Log Packet:** The relevant control and sensor variables are packed into a data structure (struct datalog) for transmission or storage.
3. **Send Data via UART:** The prepared data log packet is transmitted over UART, allowing monitoring or recording of the robot's behavior.

For TIM7 we have the following:

1. **TIM7 Interrupt:** This node is the TIM7 interrupt service routine, also triggered at a regular interval. It handles the core control logic and sensor updates for the robot.
2. **Read Line Sensors, Get Wheel Speeds:** The robot reads the line sensor values and gets the current wheel speeds.
3. **Sensor Accumulation and Oversampling:** Multiple readings from the line sensors are accumulated and processed (oversampling) to reduce noise and improve accuracy.
4. **Compute Distance from Center, Yaw error, Angular Velocity, PID Output:** The robot calculates how far it is from the center of the line, computes the yaw tracking error, estimates the robot's angular velocity, and calculates the output of the yaw PID controller.
5. **Update Wheel Set Speeds:** Using the yaw PID output and the current speed setpoint, new set speeds are assigned to the left and right wheels to steer the robot appropriately.
6. **PID Recount:** Each wheel's PID controller updates its output, based on the difference between the desired and measured wheel speeds.
7. **Update Motor PWM Duty Cycle:** The computed PID outputs are used to update the PWM signals sent to the motor drivers, controlling the actual speed of each wheel.

In summary, this flowchart provides a clear overview of the software architecture, highlighting the separation of concerns between control and logging, and illustrating how periodic tasks are efficiently managed using timer-based interrupts in an embedded system context.

7 Conclusion

We implemented a functional yaw controller for the TurtleBot. As a demonstration, we have a video of the robot finishing one loop of the test track. Since we faced the troubles described in 5.1, we lost some time, which we missed for tuning the robot in the end. That means that we had room to improve the performance of our robot.

8 Used tools

- STM32cubeIDE - flashing
- MATLAB - simulation, logging
- Visual Studio Code - code development, final report
- Python - graphs
- Github - repository here

References

- [1] SX1507/SX1508/SX1509. 2025. *Sparkfun* [online]. [accessed 2025-4-22]. Available at: <https://cdn.sparkfun.com/datasheets/BreakoutBoards/sx1509.pdf>
- [2] Pololu line sensor. 2025. *Pololu.com* [online]. [accessed 2025-4-22]. Available at: <https://www.pololu.com/docs/pdf/0J12/QTR-8x.pdf>